NVIDIA DEVELOPER Join

Technical Blog Search blog Subscribe

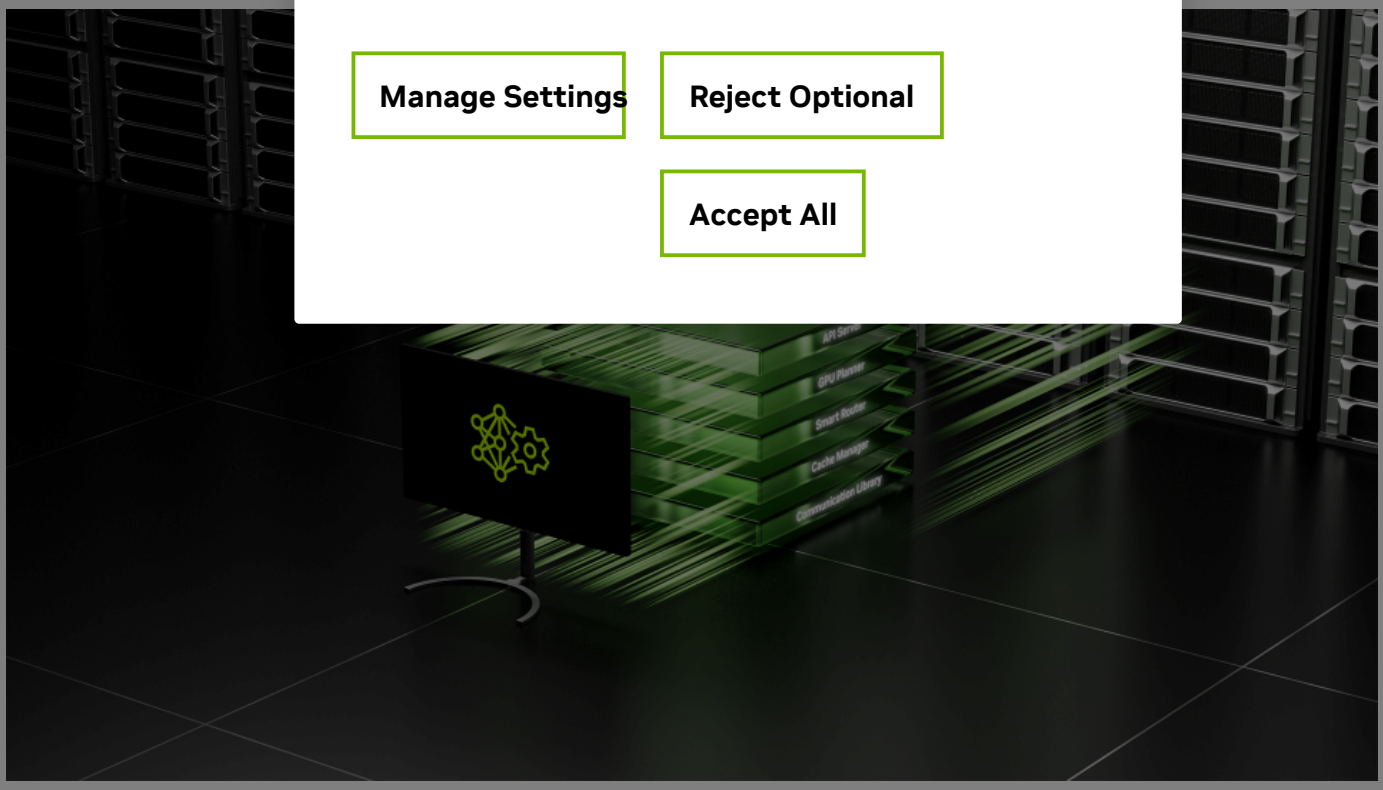
Data Center / Clou English

Deploy LM Inferer Kuberr

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Manage Settings Reject Optional

Accept All



Mar 23, 2026

+3 Like Discuss (0)

By Anish Maddipoti, Sanjay Chatterjee, Rohan Varma and Ekin Karabulut

AI-Generated Summary

compute profiles, yet traditional deployments force them onto the same hardware, leaving GPUs underutilized and scaling inflexible.

Disaggregated servi ct stages such as prefill, deco an be resourced and scale

This post will give a bernetes, explore different ec ate what they provide out of t

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

How do a ted inference

Before diving into Ku deployment modes for LLMs: In a f processes) handles the entire inference lifecycle from input to output. Disaggregated serving splits the pipeline into distinct stages such as prefill, decode, and routing, each running as independent services (see Figure 1, below).

Figure 1. Comparing aggregated and disaggregated serving

Aggregated inference

tokenizes it, runs prefill to build context, generates output tokens autoregressively (decode), and returns the response. Everything happens in a single process or tightly coupled pod group.

This is conceptually s ardware alternates between t nsive and benefits from high f dwidth-bound and benefits from la

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Disaggrega

Disaggregated archi

- Prefill workers pro ximize your GPUs for high-thr
- Decode workers g h-bound because of the au dwidth memory (HBM) ac
- Router/gateway di g between prefill and decode rkers.

Why disaggregate? Three reasons stand out:

1. Different resource and optimization profiles per stage: With disaggregation, you can match GPU resources, model sharding techniques, and batch sizes to each stage's needs rather than compromise on a single approach.
2. Independent scaling: Prefill and decode traffic patterns differ. A long-context prompt creates a large prefill burst but a steady decode stream. Scaling each stage independently lets you respond to actual demand.
3. Better GPU utilization: Separating stages lets each saturate its target resource (compute for prefill, memory bandwidth for decode) rather than alternating between both.

Frameworks like NVIDIA Dynamo and llm-d, implement this pattern. The question becomes: How do you orchestrate it on Kubernetes?

Why scheduling is the key to multi-pod inference performance on Kubernetes

directly impacts performance; placing a Tensor Parallel (TP) group's pods on the same rack with high-bandwidth NVIDIA NVLink interconnects can mean the difference between fast inference and a network bottle

- **Gang scheduling** e ntics, preventing partial
- **Hierarchical gang** oads. In disaggregated infe r role: each Tensor Parallel gro duled atomically, and the stances + router) also needs all available GPUs while the ot es but can't serve requests.
- **Topology-aware p** andwidth interconnects, mir

These three capabili ces pods based on the applica for the AI orchestration layer t example, when prefill scales independently, something needs to decide that the new pods form a gang with a minimum availability guarantee, without disrupting existing decode pods. As a result, the orchestration layer and the scheduler need to work closely for the entire application lifecycle, handling multi-level auto-scaling, rolling updates, and more, to ensure optimal runtime conditions for AI workloads.

This is where higher-level workload abstractions come in. APIs like LeaderWorkerSet (LWS) and NVIDIA Grove allow users to declaratively express the structure of their inference application: which roles exist, how they relate to each other, how they should scale, and what topology constraints matter. The API's operator translates that application-level intent into concrete scheduling constraints (including PodGroups, gang requirements, topology hints) that determine what gangs to create and when.

KAI Scheduler then plays the critical role of satisfying those constraints, solving the how: gang scheduling, hierarchical gang scheduling, and topology-aware placement. In this post, we use KAI as the scheduler, though there are other schedulers in the community that support subsets of these features. Readers can explore the broader scheduling landscape through the Cloud Native Computing Foundation (CNCF) ecosystem.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Disaggregated architectures have multiple roles each with different resource profiles and scaling needs. Since each ro pproach with LWS is to create a se

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Prefill workers (four

```
apiVersion: leader
kind: LeaderWorker
metadata:
  name: prefill-wo
spec:
  replicas: 4
  leaderWorkerTemp
    size: 2
    restartPolicy:
    leaderTemplate
      metadata:
        labels:
          role: pr
      spec:
        containers
        - name: pr
          image: <model-server-image>
          args: ["--role=prefill", "--tensor-parallel-size=2"]
          resources:
            limits:
              nvidia.com/gpu: "1"
    workerTemplate:
      spec:
        containers:
        - name: prefill
          image: <model-server-image>
          args: ["--role=prefill"]
          resources:
            limits:
              nvidia.com/gpu: "1"
```

Decode workers (two replicas, 4-degree Tensor Parallelism):

```
apiVersion: leaderworkerset.x-k8s.io/v1
kind: LeaderWorkerSet
metadata:
  name: decode-workers
spec:
  replicas: 2
  leaderWorkerTemplate:
```

```
      metadata:
        labels:
          role: decode-leader
      spec:
        containers
        - name: de
          image: <
          args: ["
          resource
            limits
              nvid
    workerTemplate
      spec:
        containers
        - name: de
          image: <
          args: ["
          resource
            limits
              nvid
```

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Router (a standard d

```
apiVersion: apps/v
kind: Deployment
metadata:
  name: router
spec:
  replicas: 2
  selector:
    matchLabels:
      app: router
  template:
    metadata:
      labels:
        app: router
    spec:
      containers:
      - name: router
        image: <router-image>
        env:
        - name: PREFILL_ENDPOINT
          value: "prefill-workers"
        - name: DECODE_ENDPOINT
          value: "decode-workers"
```

Each role is managed as its own resource. You can scale, prefill, and decode independently, and update them on different schedules.

they form a single inference pipeline. In practice, this means a few things:

- Topology coordina k for fast KV cache transfer) re across the two LWS resource
- Scaling one role d ntext requests requires fill pods aren't guaranteed inity yourself.
- Rolling out a new r endent resources—LWS's urce, but synchronizing acr

That last point is wo ys guarantee backwards compatib ecode pods on the new version r , and prefill and decode workers ronized rollout, this can crea ady but very few new prefill pods e pipeline until everything catches up.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

These patterns work. The coordination just happens outside of Kubernetes primitives: in the inference framework's routing layer, in custom autoscalers, bespoke operators, or even manually. Another option would be using Grove's API, which takes a different approach by moving that coordination into the Kubernetes resource itself.

It expresses all roles in a single PodCliqueSet:

```
apiVersion: grove.io/v1alpha1
kind: PodCliqueSet
metadata:
  name: inference-disaggregated
spec:
  replicas: 1
  template:
    cliqueStartupType: CliqueStartupTypeExplicit
    terminationDelay: 30s

    cliques:
    - name: router
      spec:
        roleName: router
```

NVIDIA DEVELOPER Join

```
      containers:
      - name: router
        image: <router-image>
        resour
          requ
            cp

  - name: prefil
    spec:
      roleName:
      replicas:
      startsAfte
      podSpec:
        schedule
        containe
        - name:
          image:
          args:
          resour
            limi
              nv
      autoScalin
        maxRepli
        metrics:
        - type:
          resour
            name: cpu
            target:
              type: Utilization
              averageUtilization: 70

  - name: decode
    spec:
      roleName: decode
      replicas: 2
      startsAfter: [router]
      podSpec:
        schedulerName: kai-scheduler
        containers:
        - name: decode
          image: <model-server-image>
          args: ["--role=decode", "--tensor-parallel-size=4"]
          resources:
            limits:
              nvidia.com/gpu: "1"
      autoScalingConfig:
        maxReplicas: 6
        metrics:
        - type: Resource
          resource:
            name: cpu
            target:
```

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

```
    topologyConstraint:
      packDomain: rack
```

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

The Grove operator r startup, and lifecycle across all of

- `startsAfter: [` startup until the router is ready ainers. When you first deploy, ro e pods start in parallel (since bo
- `autoScalingCon` e operator creates an horizon ndependently based on their ow
- `topologyConstr` k all cliques within the same ra tages over high-bandwidth in

After applying this m

```
$ kubectl get pcs,pclq,pg,pod
NAME                                                AGE
podcliqueset.grove.io/inference-disaggregated   45s

NAME                                                      AGE
podclique.grove.io/inference-disaggregated-0-router   44s
podclique.grove.io/inference-disaggregated-0-prefill  44s
podclique.grove.io/inference-disaggregated-0-decode   44s

NAME                                                     AGE
podgang.scheduler.grove.io/inference-disaggregated-0  44s

NAME                                                 READY   STATUS    AGE
pod/inference-disaggregated-0-router-k8x2m           1/1     Running   44s
pod/inference-disaggregated-0-router-w9f4n           1/1     Running   44s
pod/inference-disaggregated-0-prefill-abc12          1/1     Running   44s
pod/inference-disaggregated-0-prefill-def34          1/1     Running   44s
pod/inference-disaggregated-0-prefill-ghi56          1/1     Running   44s
pod/inference-disaggregated-0-prefill-jkl78          1/1     Running   44s
pod/inference-disaggregated-0-decode-mn90p           1/1     Running   44s
pod/inference-disaggregated-0-decode-qr12s           1/1     Running   44s
```

One PodCliqueSet, three PodCliques (one per role), one PodGang for coordinated scheduling, and pods matching each role's replica count. The `startsAfter` dependency is enforced through

Scaling d

Once a disaggregate challenge.
Prefill and decode ha l workers
based on time to firs cy (ITL)
independently, to me s.

In practice, disaggre

1. **Per-role scaling:** ad 4 to 6 replicas)
2. **Per-TP-group scali** ou can't add half a TP group.
3. **Cross-role coordina** router to handle increased th

Different tools addre

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

How inference frameworks coordinate scaling

Inference frameworks address scaling at the application level with custom autoscalers that have visibility into inference-specific metrics. llm-d's workload variant autoscaler (WVA) monitors per-pod KV cache utilization and queue depth via Prometheus, using a spare-capacity model to determine when replicas should be added or removed. Rather than scaling deployments directly, WVA emits target replica counts as Prometheus metrics that standard HPA/Kubernetes-based event-driven autoscaling (KEDA) act on—keeping the scaling actuation within Kubernetes-native primitives.

The NVIDIA Dynamo planner takes a different approach: It natively understands disaggregated serving, running separate prefill and decode scaling loops that target TTFT and ITL SLAs respectively. It predicts upcoming demand using time-series models, computes replica requirements from profiled per-GPU throughput curves, and enforces a global GPU budget across both roles.

output has nowhere to go—decode bottlenecks and KV cache transfer queues up. Application-level autoscalers handle this because they can see the full pipeline; Kubernetes-native HPA targeting individual r

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Scaling wit

With one LWS per ro

```
kubectl scale lws
kubectl scale lws
```

Standard HPA can ta ynamo planner or llm-d's au coordination logic lives in the auto

Scaling wit

Grove brings per-role lica count and optional `autoScalingConfig`, so HPAs can manage roles independently based on per-role metrics:

```
kubectl scale pclq inference-disaggregated-0-prefill --replicas=6
```

The operator creates additional prefill pods while leaving the router and decode untouched:

```
podclique.grove.io/inference-disaggregated-0-decode   5m

NAME
pod/inference-disa
pod/inference-disa
pod/inference-disa
pod/inference-disa
pod/inference-disa
pod/inference-disa
pod/inference-disa
pod/inference-disa
pod/inference-disa
pod/inference-disa
```

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Six prefill pods, two

For roles that use mu ensures multiple PodCliques een them. For example, in a configu nd four worker pods:

```
podCliqueScalingGroups:
   - name: prefill
     cliqueNames: [pleader, pworker]
     replicas: 2
     minAvailable: 1
     scaleConfig:
       maxReplicas: 4
```

With replicas: Two, this creates two complete prefill instances: two x (one leader + four workers) = 10 pods total. The `minAvailable: One` guarantee means the system won't scale below one complete Tensor Parallel group.

Scaling the group from two to three replicas adds a third complete instance while preserving the 1:4 leader-to-worker ratio:leader-to-worker ratio:

```
$ kubectl scale pcsg inference-disaggregated-0-prefill --replicas=3
```

Both the leader and worker cliques scaled together as a unit, the new replica (prefill-2) has one `pleader` pod and four `pworker` pods, matching the ratio. A new PodGang was created for the third replica to ensure it gets gang-scheduled.

```
NAME                                                                AGE
podclique.grove.io/inference-disaggregated-0-prefill-0-pleader      10m
podclique.grove.io
podclique.grove.io
podclique.grove.io
podclique.grove.io
podclique.grove.io

NAME
podgang.scheduler.
podgang.scheduler.
podgang.scheduler.
```

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Getting s

Whether you're runn your cluster, the building blocks f e open. Each approach in this blog city and integrated coordinat

The right choice depends on your workload, your team's operational model, and how much lifecycle management you want the platform to handle versus the application layer.

Check out these resources for more information.

- NVIDIA Grove
- KAI Scheduler
- NVIDIA Dynamo

Join us at Kubecon EU

If you're attending KubeCon EU 2026 in Amsterdam, drop by at booth No. 241 and join the session where we will cover an end-to-end open source AI inference stack. Explore the Grove Deployment Guide and ask questions on GitHub or Discord. We'd love to hear how you're thinking about disaggregated inference on Kubernetes.

Discuss (0) +3 Like

Tags

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

About the Au

tly works on
VIDIA Grove.
quired by
an II program

works on
g to enable AI
or and
architect of the open source NVIDIA Grove project. Previously he worked on multiple DoE/DARPA funded advanced technology projects towards designing the first exascale systems. His interests include novel programming models, parallel languages, and runtime systems.

View all posts by Sanjay Chatterjee >

About Rohan Varma

Rohan Varma is an AI dev tech engineer at NVIDIA. He focuses on optimizing NVIDIA inference solutions including Dynamo, Grove, and TensorRT-LLM. He has a master's degree in Computer Science from University of Michigan, Ann Arbor. He enjoys racing games, piano, and most racket sports.

View all posts by Rohan Varma >

About Ekin Karabulut

Ekin Karabulut is a data scientist and developer advocate previously at Run:ai, now at NVIDIA, exploring the efficient usage of large models in different production scenarios. Previously she

GPU usage in research and industry settings. She established the AI Infrastructure Club and is based in Munich, Germany.

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Comments

Start the d

Related posts

Enhancing Distributed Inference Performance with the NVIDIA Inference Transfer Library

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

Enabling Hori

Kubernetes

s on

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA Dynamo Accelerates llm-d Community Initiatives for Advancing Large-Scale Distributed Inference

NVIDIA DEVELOPER

Join

NVIDIA and our third-party partners use cookies and other tools to collect and record information you provide as well as information about your interactions with our websites for performance improvement, analytics, and to assist in our marketing efforts. By clicking "Accept All", you consent to our use of cookies and other tools as described in our Cookie Policy. You can manage your cookie settings by clicking on "Manage Settings". Please see our Privacy Policy for more information on our privacy practices.

NVIDIA Dynar ... on, and Networking O

The Next Generation of AI Begins

It all starts here. June 1–4 | Taipei

Register Now

NVIDIA GTC

Sign up for NVIDIA News

Subscribe

Follow NVIDIA Developer

NVIDIA DEVELOPER

Privacy Policy | Your Privacy Choices | Terms of Use | Accessibility | Corporate Policies | Contact

Copyright © 2026 NVIDIA Corporation